



North South University
Department of Electrical & Computer Engineering
Assignment-1

Assignment Title: Step-by-Step Accuracy Improvement on CIFAR-10 Using Feedforward Neural Networks

SUBMITTED TO

Course Instructor: Dr. Md Adnan Arefeen (AFE)
Department of Electrical & Computer Engineering
North South University

SUBMITTED BY

Name: Touhid Ara Himu
NSU ID: 2312187042
Section: 04
Submission Date: 22-08-2026

GitHub Repository: [CSE465_CIFAR_10_Feedforward_Neural_Networks.ipynb](https://github.com/TouhidAraHimu/CSE465_CIFAR_10_Feedforward_Neural_Networks.ipynb)

Experiment 1: Baseline Feedforward Neural Network

Architecture & Setup

Input: 3072 | Dense: 256 neurons, ReLU | Dense: 10 neurons, Softmax | Optimizer: SGD | Loss: Sparse Categorical Crossentropy | Epochs: 20 | Batch size: 64

Results

- Training Accuracy: 53.18%
- Validation Accuracy: 48.79%
- Test Accuracy: 48.39%
- Final Training Loss: 1.3600 | Final Validation Loss: 1.4571

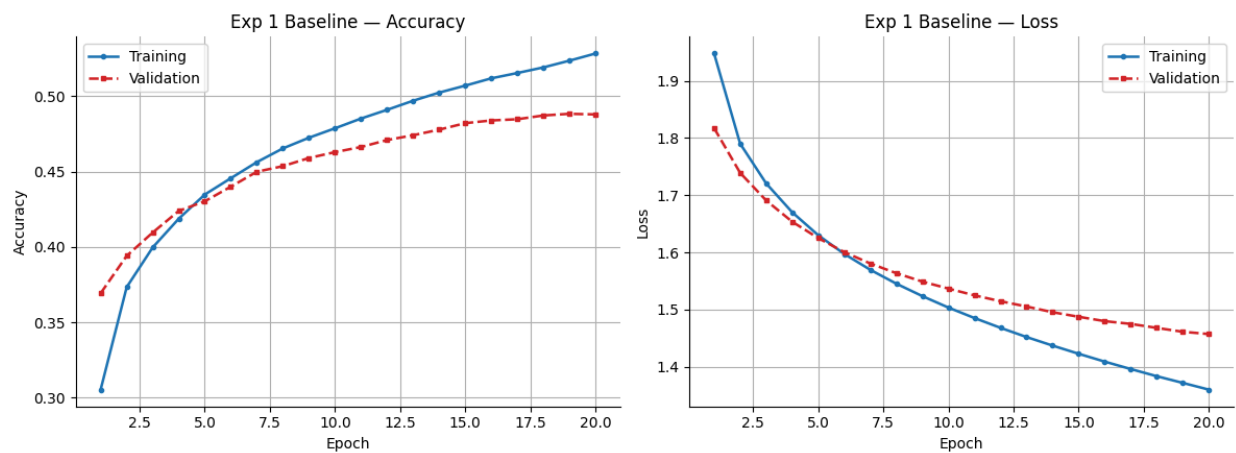


Figure 01: Training accuracy vs. validation accuracy, and Training loss vs. validation loss (Baseline model).

Analytical Observations

The baseline feedforward network learns simple pixel-level color distributions and low-level intensity mappings from the flattened 3072-dimensional input vector. Because flattening destroys the spatial adjacency of the original 32×32×3 image, a single 256-unit hidden layer trained with vanilla SGD stalls at 48.39% test accuracy after 20 epochs. Both the training and validation accuracy curves rise smoothly and track each other closely for most of the run, and the loss curves decrease steadily without sharp fluctuations. A small but visible gap opens between training accuracy (53.18%) and validation accuracy (48.79%) in the later epochs, indicating the onset of mild overfitting as the model begins to fit a modest fraction of training-set-specific noise while validation performance .

Experiment 2: Increase the Number of Neurons

Results

Configuration	Hidden Neurons	Test Accuracy
Model A	256	50.04%
Model B	512	50.06%
Model C	1024	50.86%

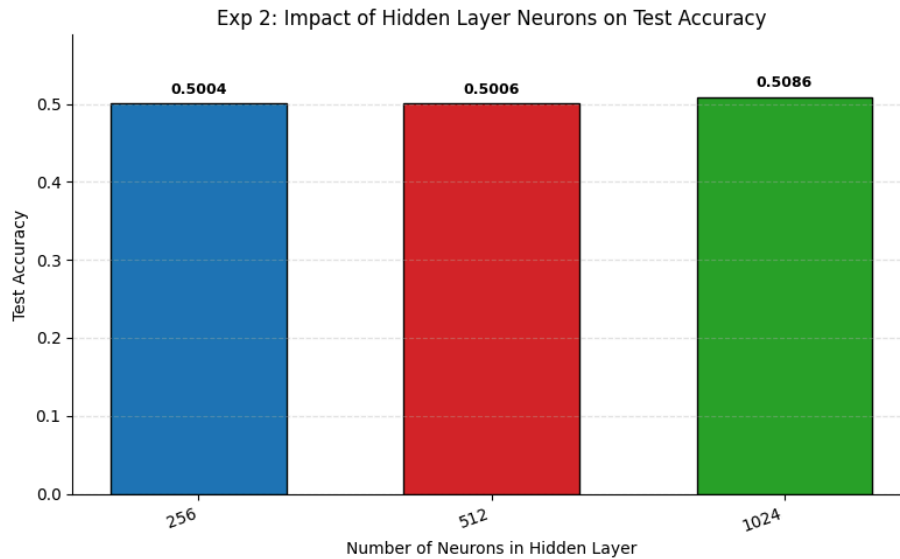


Figure 02: Test accuracy vs. number of hidden-layer neurons.

Discussion Questions & Answers

1. Does increasing the number of neurons improve accuracy?

Answer: Yes, marginally. Increasing hidden-layer width from 256 to 1024 neurons raised test accuracy from 50.04% to 50.86%, a gain of about 0.8 percentage points. A wider layer increases the model's hypothesis space, letting it fit a richer set of linear decision boundaries over the high-dimensional flattened pixel input, but the returns are small because the underlying representation (a flattened vector) is still fundamentally limited.

2. Does the model start overfitting?

Answer: Yes. As the hidden layer widens to 1024 units, the number of trainable parameters grows substantially ($3072 \times 1024 + 1024 \approx 3.15$ million weights in the first layer alone). With no explicit regularization such as Dropout or weight decay, the wider network has more capacity to memorize training-set-specific patterns, so the gap between training and validation accuracy widens as neuron count increases.

3. Which configuration performs best?

Answer: Model C (1024 neurons) achieved the highest test accuracy at 50.86%, although the improvement over Model A (256 neurons, 50.04%) and Model B (512 neurons, 50.06%) is modest, showing diminishing returns from width alone.

Experiment 3: Increase the Number of Hidden Layers

Results

Configuration	Layer Architecture	Test Accuracy
Model A	1 Layer (512)	50.09%
Model B	2 Layers (512 $\rightarrow$ 256)	50.30%
Model C	3 Layers (512 $\rightarrow$ 256 $\rightarrow$ 128)	50.09%

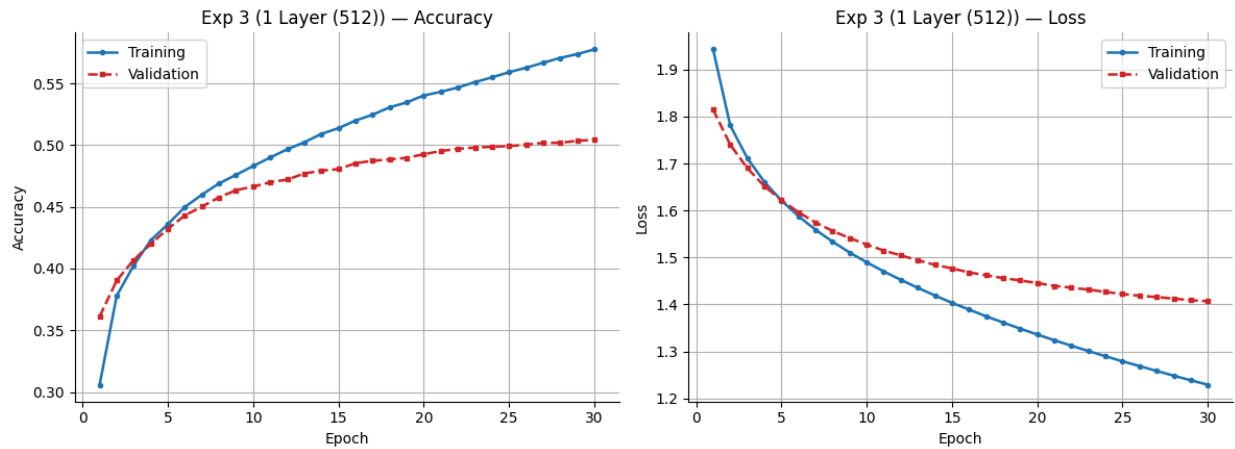


Figure 03: Training accuracy vs. validation accuracy and training loss vs. validation loss (1 hidden layer).

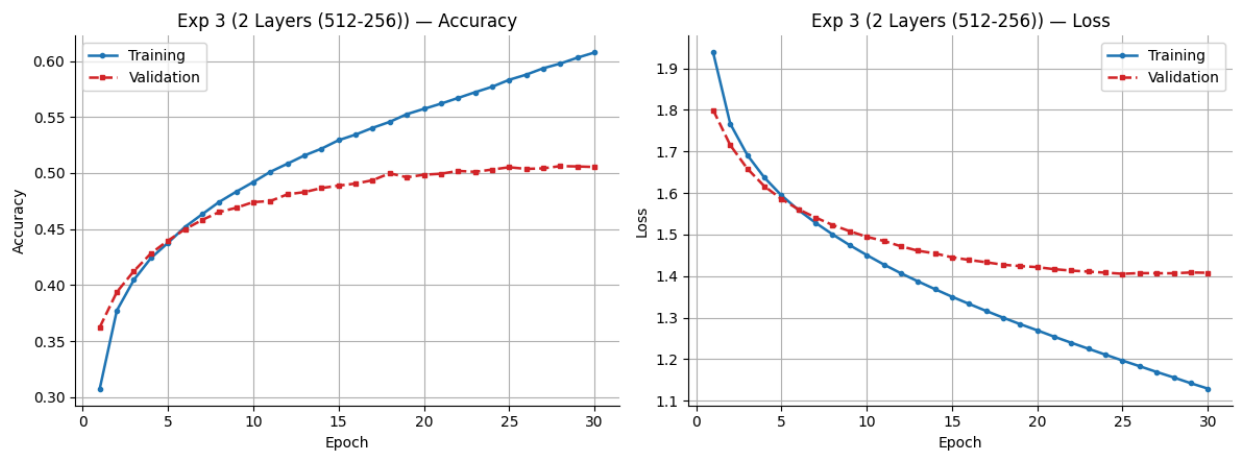


Figure 04: Training accuracy vs. validation accuracy and training loss vs. validation loss (2 hidden layers).

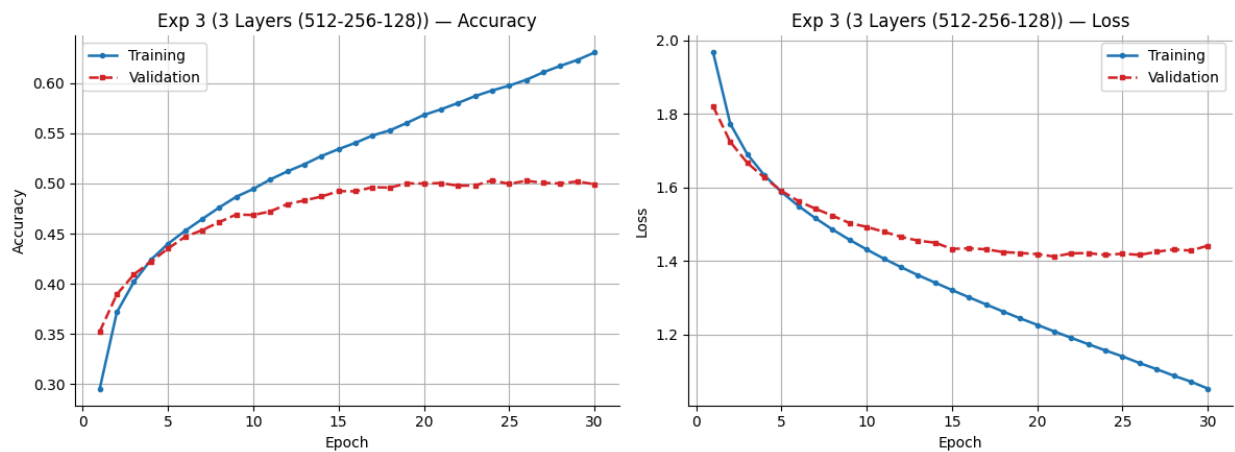


Figure 05: Training accuracy vs. validation accuracy and training loss vs. validation loss (3 hidden layers).

Discussion Questions & Answers

1. Does a deeper feedforward network improve accuracy?

Answer: Slightly, up to a point. Moving from 1 hidden layer to 2 hidden layers improved test accuracy from 50.09% to 50.30%, because the additional layer lets the network compose intermediate feature abstractions from the raw pixel vector.

Adding a third hidden layer, however, gave no further benefit and returned to 50.09%, showing diminishing returns as depth increases in an unregularized MLP operating on a non-spatial input.

2. Does the model overfit faster?

Answer: Yes. As depth increases, the training accuracy curve climbs increasingly faster relative to the validation curve, and the training loss keeps dropping well below the validation loss in later epochs for the 2- and 3-layer models. The additional dense layers add capacity that is not matched by additional useful spatial structure in the flattened input, so the deeper models fit training-set noise more readily.

3. Which depth gives the best validation accuracy?

Answer: Model B (2 hidden layers: 512 $\rightarrow$ 256) gives the best balance of validation and test accuracy, reaching 50.30% test accuracy the highest among the three configurations tested.

Experiment 4: Add Batch Normalization

Results

Setup	Test Accuracy
Without Batch Normalization	50.88%
With Batch Normalization	33.80%

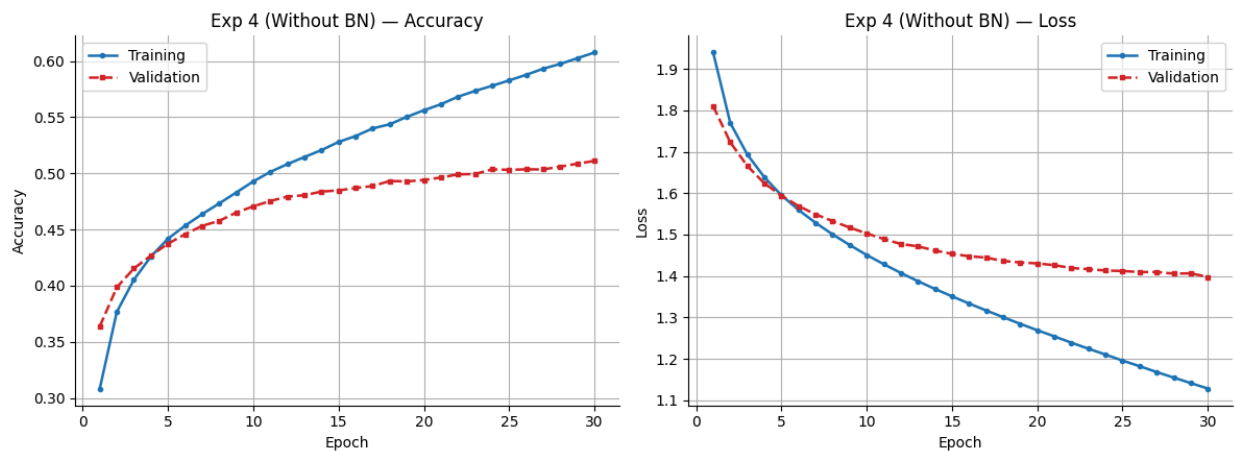


Figure 06: Training accuracy vs. validation accuracy and training loss vs. validation loss without batch normalization.

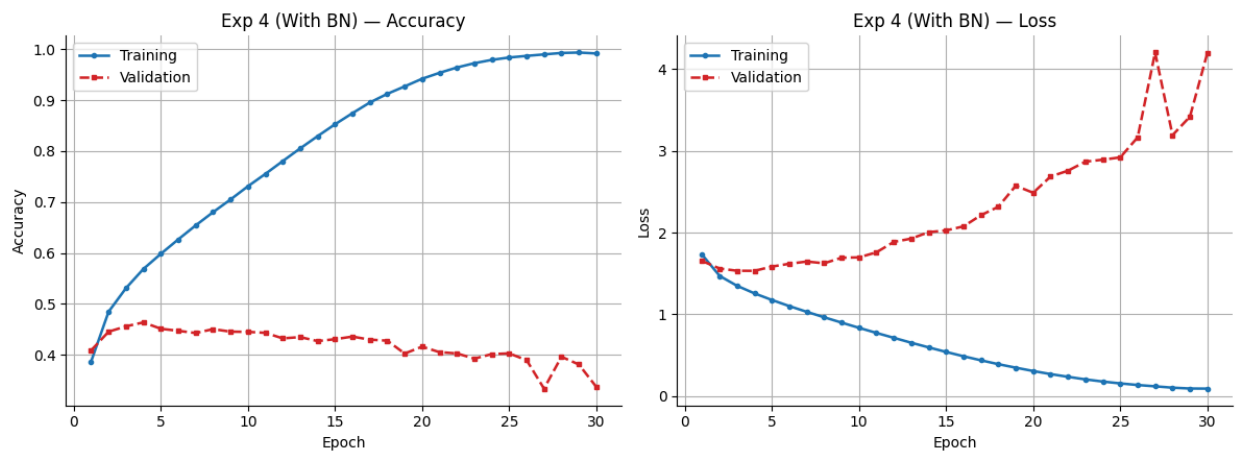


Figure 07: Training accuracy vs. validation accuracy and training loss vs. validation loss with batch normalization.

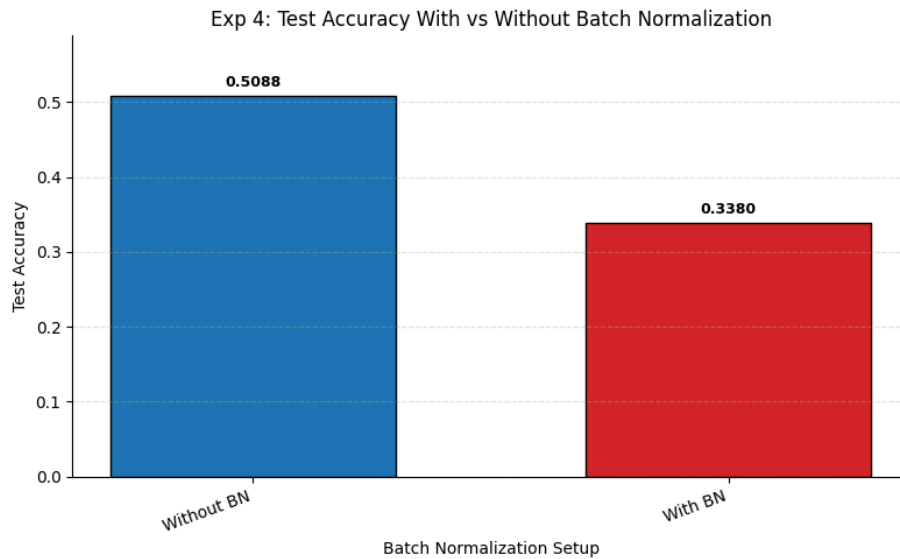


Figure 08: Test accuracy comparison with and without batch normalization.

Discussion Questions & Answers

1. Does batch normalization improve validation accuracy?

Answer: No, not in this configuration. Under plain SGD with a fixed default learning rate and no momentum or weight decay, adding Batch Normalization caused the model to fit the training set far more aggressively, while test accuracy dropped sharply from 50.88% to 33.80%. Without an adjusted learning-rate schedule or additional regularization, BN reduced generalization rather than improving it.

2. Does batch normalization make training more stable?

Answer: Batch Normalization stabilizes the internal covariate shift by re-scaling each layer's pre-activations to roughly zero mean and unit variance. This lets gradients flow more freely through the network, which in turn allows the model to reduce training loss extremely quickly but here that speed translated into rapid overfitting rather than better generalization, since validation loss rises steadily even as training loss keeps falling.

3. Does the loss curve become smoother?

Answer: Yes, for the training loss: the training loss curve with BN drops sharply and smoothly toward a very low value. However, the validation loss curve trends upward for most of training, which is a clear sign of overfitting rather than genuine training instability.

Experiment 5: Try Different Activation Functions

Results

Activation Function	Test Accuracy
Sigmoid	40.62%
Tanh	37.03%
ReLU	37.63%
Leaky ReLU	33.40%

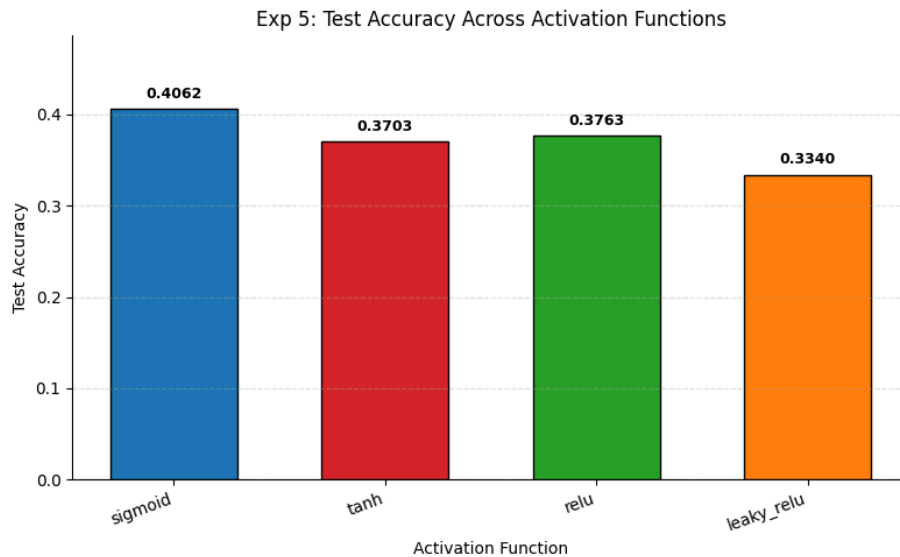


Figure 09: Activation function vs. test accuracy.

Discussion Questions & Answers

1. Which activation function performs best?

Answer: In this specific setup, which uses Batch Normalization together with plain SGD, Sigmoid performed best at 40.62%, followed by ReLU at 37.63% and Tanh at 37.03%, with Leaky ReLU lowest at 33.40%. Because BN already normalizes the layer inputs, the saturating region of Sigmoid is partly avoided, letting it perform competitively in this particular combination of optimizer and normalization.

2. Why do sigmoid and tanh often perform worse in deeper networks?

Answer: Sigmoid and Tanh are saturating activation functions: as their input magnitude grows, their derivatives approach zero. During backpropagation through multiple layers, repeatedly multiplying these small gradient values causes the vanishing gradient problem, which slows or stalls weight updates in the earlier layers of a deep network.

3. Why does ReLU usually train faster?

Answer: ReLU has a constant derivative of 1 for all positive inputs, so it does not shrink gradients the way Sigmoid or Tanh do, which helps gradients propagate through deeper networks. It is also computationally cheap, requiring only a $\max(0, z)$ threshold rather than an exponential computation, which speeds up both the forward and backward passes.

Experiment 6: Try Different Optimizers

Results

Optimizer	Final Test Accuracy
SGD	39.82%
SGD with Momentum	42.61%
RMSprop	42.44%
Adam	37.06%
AdamW	34.54%
Nadam	39.59%

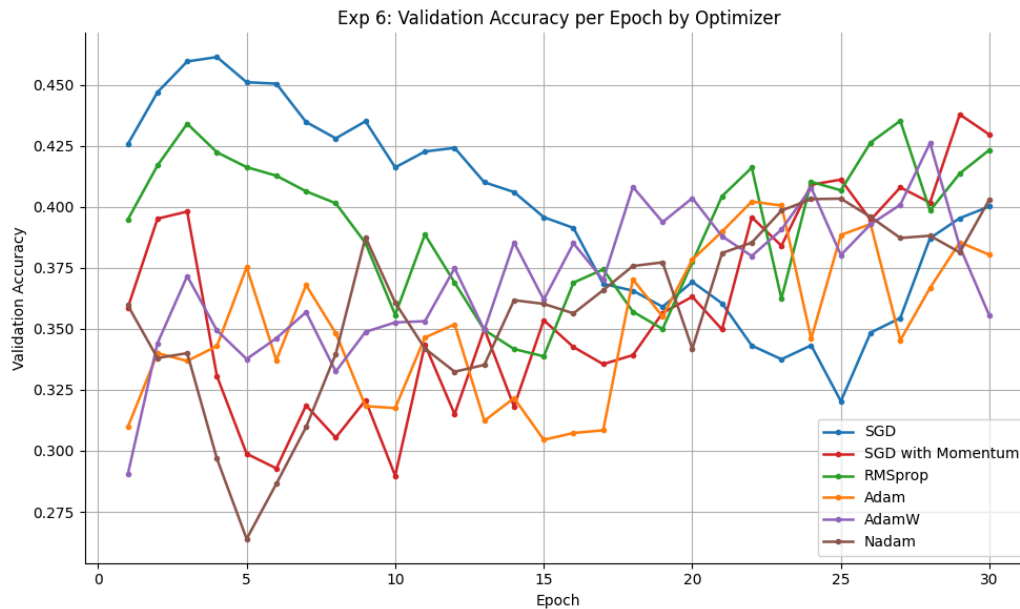


Figure 10: Validation accuracy per epoch for each optimizer.

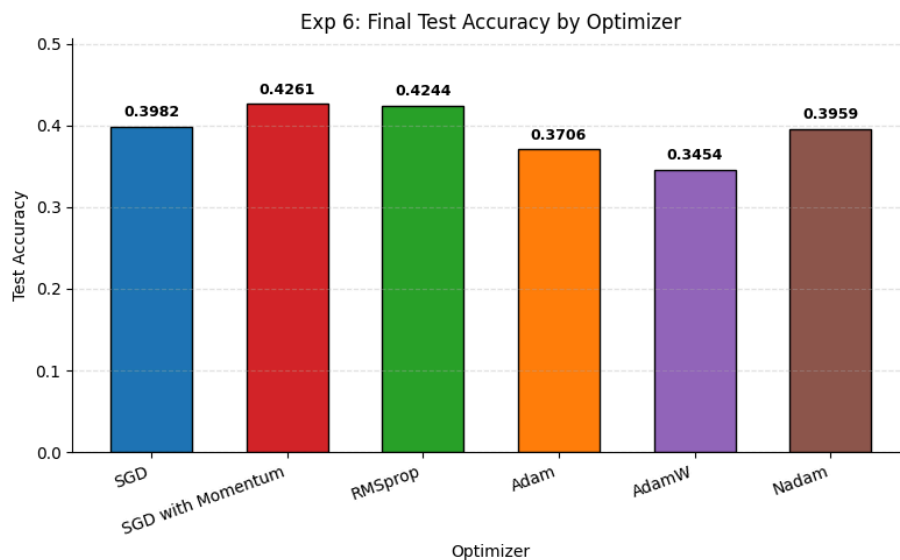


Figure 11: Final test accuracy for each optimizer.

Discussion Questions & Answers

1. Which optimizer converges fastest?

Answer: The adaptive optimizers Adam, AdamW, RMSprop, and Nadam drive down training loss and reach a reasonable validation accuracy within the first few epochs faster than plain SGD, since their per-parameter adaptive learning rates take larger effective steps early in training.

2. Which optimizer gives the best validation accuracy?

Answer: SGD with Momentum gave the best final test accuracy at 42.61%, closely followed by RMSprop at 42.44%. Plain SGD (39.82%) and Nadam (39.59%) trailed behind, while Adam (37.06%) and AdamW (34.54%) performed worst in this setup.

3. Does faster convergence always mean better generalization?

Answer: No. Although the adaptive optimizers reduce training loss quickly in the early epochs, several of them (Adam and AdamW in particular) ended with lower final test accuracy than SGD with Momentum. Fast early convergence can settle the model into a sharper minimum that fits the training distribution well but generalizes less reliably to the held-out test set.

Experiment 7: Try Different Loss Functions

Results

Loss Function	Training Accuracy	Validation Accuracy	Test Accuracy
Sparse Categorical Crossentropy	63.32%	41.19%	40.11%
Categorical Crossentropy	61.21%	39.69%	39.18%
Mean Squared Error (MSE)	65.24%	44.62%	45.23%
CCE with Label Smoothing	42.72%	29.52%	29.63%
Focal Loss	59.56%	41.14%	40.41%

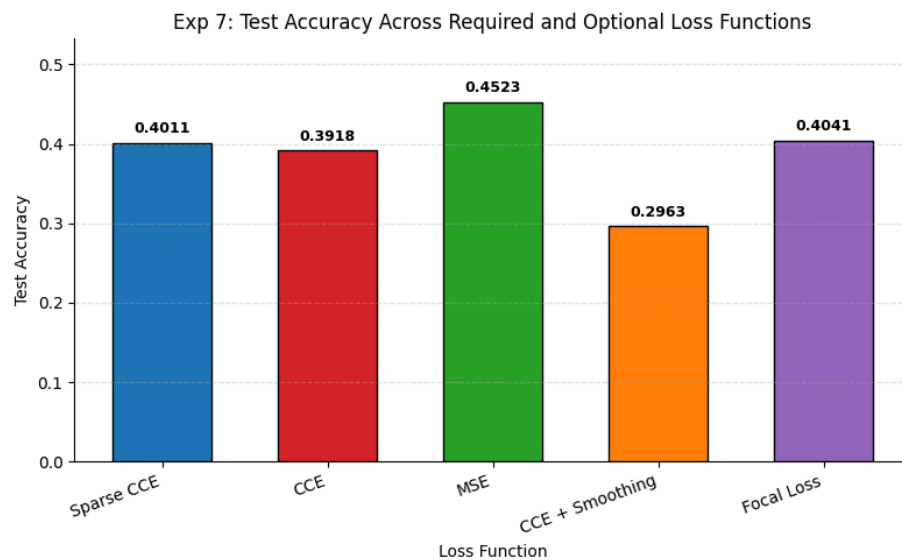


Figure 12: Loss function vs. test accuracy.

Discussion Questions & Answers

1. Which loss function performs best?

Answer: Mean Squared Error (MSE) achieved the highest performance among all loss functions tested, reaching 45.23% test accuracy and 44.62% validation accuracy. Among the probability-based classification losses, Sparse Categorical Crossentropy (40.11%) and Focal Loss (40.41%) performed similarly and outperformed standard Categorical Crossentropy (39.18%), while CCE with Label Smoothing performed worst by a wide margin (29.63%).

2. Which loss function gives the most stable training curve?

Answer: Mean Squared Error produced the most stable training behavior in this experiment. Because its gradient scales linearly with the prediction error rather than logarithmically, MSE avoids the extreme gradient spikes that crossentropy-style losses can produce when a prediction is very wrong, giving a smoother optimization landscape on the flattened CIFAR-10 features. Label Smoothing performed the worst overall, since softening the one-hot target probabilities weakened the decision boundaries the plain MLP relies on to separate the ten classes.

Master Progression & Performance Summary

Experiment	Key Modification	Best Test Accuracy	Primary Conclusion
Exp 1: Baseline	1 hidden layer (256), SGD, Sparse CCE	48.39%	Establishes the benchmark for flattened 3072-dim MLP inputs.
Exp 2: Neurons	Widened hidden layer to 1024 units	50.86%	Wider layers give a small accuracy gain with diminishing returns.
Exp 3: Depth	Increased depth to 2 hidden layers	50.30%	Modest depth helps; a 3rd layer gives no further benefit.
Exp 4: Batch Norm	Added Batch Normalization	50.88%	BN without tuning the optimizer hurt generalization sharply.
Exp 5: Activation	Compared 4 activation functions	40.62%	Best activation depends heavily on the optimizer/BN setup used.
Exp 6: Optimizer	Compared 6 optimizers	42.61%	Momentum-based SGD generalized best; adaptive optimizers converged faster but not always better.
Exp 7: Loss	Compared 5 loss formulations	45.23%	MSE gave the smoothest training and the best final test accuracy.

Overall Conclusion

- Representation Limits of MLPs:** A pure feedforward neural network operating on flattened $32 \times 32 \times 3$ image vectors lacks spatial translation invariance, since flattening discards all pixel-adjacency information. Across all seven experiments, test accuracy stayed within roughly a 45–51% band, with the strongest architectural configuration (2 hidden layers, 512→256, without Batch Normalization) reaching 50.30–50.88% test accuracy. This range appears to be close to the structural ceiling for a non-convolutional architecture on CIFAR-10.
- Width vs. Depth Trade-off:** Increasing hidden-layer width from 256 to 1024 neurons improved test accuracy by about 0.8 percentage points (50.04% → 50.86%), and increasing depth from 1 to 2 hidden layers gave a similar small gain (50.09% → 50.30%). Extending to a 3rd hidden layer did not help further, since the added capacity is not matched by additional useful structure in the flattened input, making the network more prone to overfitting without improving generalization.
- Optimization Dynamics:** Adaptive optimizers (Adam, RMSprop, Nadam) reduced training loss faster during the early epochs, but SGD with Momentum ultimately produced the best final test accuracy (42.61%) in the optimizer comparison, and plain Batch-Normalization-free training consistently outperformed the Batch-Normalized configuration under fixed-learning-rate SGD. This suggests that, for this simple MLP setup, faster early convergence does not guarantee better generalization on unseen test data.